Effort Estimation Models Using Evolutionary Learning Algorithms for Software Development

Goldie Gabrani
Department of Computer Engineering
BML Munjal University
Gurgaon, India
goldie.gabrani@bml.edu.in

Neha Saini
Department of Computer Engineering
BML Munjal University
Gurgaon, India
neha.saini@bml.edu.in

Abstract— **Software effort estimation is a complicated task being carried out by software developers as very little information is available to them in the early phases of software development. The information collected about various attributes of software needs to be subjective, which otherwise can lead to uncertainty. Inaccurate software effort estimation can be disastrous as both underestimation and overestimation may result in schedule overruns and incorrect estimation of budget. This paper focuses on the comparative study of various non-algorithmic techniques used for estimating the software effort by empirical evaluation of five different evolutionary learning algorithms. The accuracy of these algorithms is found out and the behavior of these algorithms is analyzed with respect to the size and the type of data. All the five techniques are applied on three different datasets and various paramenters such as MMRE, PRED(25), PRED(50), PRED(75) are calculated. The proposed results are compared to other machine learning methods like SVR, ANFIS etc. The results show that evolutionary learning algorithms give more accurate results than machine learning algorithms.**

Keywords— ***Software Effort Estimation; Evolutionary Learning Algorithms; Machine Learning Algorithms; Mean Magnitude of the Relative Error MMRE; Percentage of Predictions(x) PRED(x);***

I. INTRODUCTION

Software effort estimation is one of the most crucial activities that aids in software project planning and in prediction of amount of time and cost required for a particular software project [1]. The concept of software effort estimation first appeared in 1950's and after that it has caught the attention of software developers who wants to make accurate estimations for software effort [2][3]. Since that time various models have been proposed for predicting software effort, but a lot of problems are associated with them because data related to software projects available in the beginning is normally incomplete and inconsistent. It is difficult to determine which technique is more suitable on which dataset and which one gives more accurate results. Till date there is no unique software effort estimation model which can give fast, correct and accurate software effort predictions under all circumstances. The challenge most of the software organizations face is to deliver the software within budget and on time with desired functionalities. Both under estimation and over estimation have negative impact on software development. Under estimation can lead to delay in schedules and cost overruns which in turn reduce the quality of delivered products. Whereas, over estimation can lead to loss of customers and partners. Therefore, for any software organization, accurate estimation of effort becomes very important task for successful completion of a software project within the given budget and timelines. Various Software Estimation techniques have been proposed in the literature. These techniques broadly fall under the following categories: Algorithmic and Non-Algorithmic.

Algorithmic techniques estimate software effort based on mathematical equations comprising of many variables. Most of these methods are based on regression technique and provide accurate estimates when similar historical project exists [4]. However, these methods need to be adjusted according to local conditions [4], have difficulty in modelling complex set of attributes, do not support data that is divided in categorical way [6], requires a large dataset, no missing data items, requires predictor variables to be not correlated, have strong sensitivity towards outliers and make use of size attribute which is very difficult to estimate in the beginning of software development lifecycle. Analysis of data using these methods therefore is quite complex [7, 8]. The main algorithmic techniques belong to two broad categories depending on whether Line of Code (LOC) or Functional Point Analysis (FPA) is measured for the purpose of calculation of effort and they include techniques like COCOMO, SLIM, RCA PRICE-S, SEER-SEM and ESTIMACS [1-3].

Non-Algorithmic techniques, on the other hand does not necessarily are based on mathematical relations**.** One of the popular Non-Algorithmic techniques is based on judgement of experts. However, the estimations done by experts are sometimes questionable, may vary and be inconsistent and lack repeatability as they are highly dependent on human memory [2]. These drawbacks have led to the development of other Non-Algorithmic techniques and most of these techniques are centered on evolutionary learning methods like Neural Networks, Genetic Programming, Fuzzy logic, Rule based induction etc. Research is being done to formulate techniques based on these evolutionary algorithms for the estimation of effort [9]. However, these algorithms are not widely deployed in the area for software effort estimation even though they have shown outstanding performance in various other fields like weather prediction, speech recognition, digital signature verification, forecasting of stock values, turnover calculation, image processing, speech recognition, signature verification, medicine, etc. These approaches are based on learning from previous estimation experience and they try to automate the

978-1-5090-0669-4/16/$31.00 © 2016 IEEE

estimation process by training themselves using historical data from previous experience to produce better results by building computerized models [2]. These models keep on refining themselves as and when new data is provided over time and are capable of learning incrementally by adjusting their algorithmic parameters in order to bring the difference between the known actual values and predicted values within some predefined delta. The delta value has to be specified in a manner that the system should not become over trained to the known previous historical data. They can model complex set of relationships between attributes very easily, insensitive to outliers and generally deal with fields which are understood very poorly. But these methods are sensitive to the data on which they are trained.

The evolutionary learning methods have become popular in recent years as they try to imitate natural evolution while finding solution and tend to refine the solution until an optimal one is achieved. Different evolutionary learning techniques have some apparent design differences but also have some common features. In order to get best features of these different algorithms, different categories of evolutionary learning algorithms can be hybridized. In this paper,five algorithms have been used, out of which three are variants of hybridization of genetic programming and fuzzy learning algorithms viz. GFS-GPG-R, GFS-GSP-R, GFS-SAP-Sym-R, fourth one involves ensembling of neural networks, Ensemble-R and the fifth one involves fuzzy random set based modeling, FRSBM. These algorithms have been applied for estimation of software effort and empirical study of these methods on well-known datasets has been done. For this purpose, publicly available datasets of three different sizes have been taken. Range of sizes include small (Miyazaki94 dataset), medium (Maxwell dataset) and large (Desharnais dataset). The, empirical evaluation is done to find out MMRE andPRED(25). The results are then analyzed to find which algorithm works best for which dataset. This work is of utmost importance for software organizations because it helps them to find out which algorithm is best suited for which type of dataset, and then the prediction can be done in a much more accurate manner. Accurate estimation helps software organizations to make precise calculations of budget and manpower requirements of different projects. After analyzing the results by using different evolutionary algorithms on the datasets we conclude that different datasets show different results with different algorithms. The performance of a particular evolutionary algorithm depends on the type of data on which our evolutionary algorithm is trained. As per our studies, GFS-SAP-Sym-R is a good method for dataset of any size (small, medium, large). However GFS-GPG-R gives the best performance on a large dataset, whereas GFS-GSP-R and GFS-SAP-Sym-R give the best performance when the dataset is medium and small respectively. The Ensemble-R method has the worst performance irrespective of the size of the dataset. This clearly reflects that there is no unique software effort estimation technique which can give accurate results under all circumstances and there exists a strong dependency between the type of data and software effort estimation technique being used.

The rest of the paper is organized as follows. Section 2 provides the details of research background. In section 3, we briefly explain the research methodology. Section 4 contains empirical evaluation results. Finally the paper is concluded in section 5.

II. RESEARCH BACKGROUND

Five evolutionary learning algorithms namely FRSBM, GFS-GPG-R, GFS-GSP-R, GFS-SAP-Sym-R, and Ensemble-R have been used in the paper. With the help of FRSBM, a mathematical model can be built which takes into account the uncertainties/fuzziness in the inputs and treats inputs as random processes. Ithas been seen in the literature that combination of genetic programming and fuzzy logic gives good results for complex real life problems and hence can also be used for software effort estimation**.**In GFS-GPG-R algorithm, genetic programming grammar operators like cross over and mutation are used to optimize the membership function values and rule base is created by fuzzy logic. In GFS-GSP-R algorithm, symbolic fuzzy learning is based on genetic programming and simulated annealing. Here both genetic programming and simulated annealing are combined to find more optimal value for membership function and rule base of fuzzy control systems. Genetic algorithm runs parallel on a set of solutions and information is exchanged using crossover operation unlike simulated annealing which works on one solution at a time. While Genetic algorithm uses the same selection strategy during run of an algorithm, simulated annealing can vary parameters to evaluate the solution. This way the advantages of both methods can be taken.In GFS-SAP-Sym-R, similar to GFS-GSP-R, both genetic algorithm and simulated annealing are combined. Additionally, it has application dependent crossover operators which can be applied to different search schemes allowing them to be applied to new fields. Also it can perform automatic feature sub selection. In Ensemble- R, many neural networks are joined together to solve a particular problem.In this way errors due to different neural networks average out and there is an improvement in generalization capability.

III. RESEARCH METHODOLOGY

The methodology deployed in this paper to estimate the effort broadly consists of three steps. First, data is pre-procesed to remove noisy data. This is done to obtain a clean dataset asmost of theeffort estimation datasets consists of missing values. For example, Desharnais dataset used in this paper has missing values for projects numbered 38, 44, 65 and 75. In the proposed work, data is pre-processedby using KNN missing value (K-Nearest Neighbour) algorithm. Data that comes out after this step is the filtered data. Many variants of KNN missing value algorithm have been reported in the literature. The variant used in this paper is the weighted average of K-nearest neighbours weighted by the inverse of distance [11]. After pre-processing, the filtered data is trained and validated using five different evolutionary techniques viz. GFS-GPG-R, GFS-SAP-Sym-R and GFS-GSP-R, Ensemble-R and FRSBM to predict effort. The method used for training and validation in the proposed work is called k-cross validation. In this paper k is assumed to be 10. The statistical library of KEEL is used then to analyze and compare the results of various algorithms. It provides user friendly graphical user interface where multiple functional nodes can be combined to form a single process.

After second step, different performance measures are calculated. The performance measures used for evaluation of software effort are MMRE and PRED(25) [12]. These measures have been chosen as most of the studies done in this area have used them for software prediction systems. All of these are based on Magnitude of Relative Error (MRE), MRE is partial towards the underestimations. Mean Magnitude of the Relative Error (MMRE) and percentage of predictions within 25% (PRED(25)) of the true value have been calculated by the formulas 1 through 3. The pseudocode for estimating software effort is given in Figure 3.1.

MMRE is to be minimized whereas PRED(25) is to be maximised.Results are analysed to see which evolutionary algorithms is better for which dataset.

MMRE value can be calculated for any dataset consisting of m observations in following manner:

MMRE=1/m (summation of MRE from 1 to m) (1)

Where MREi is difference between actual and predicted values divided by the actual values. It is calculated as follows:

MREi= (A-P) / A (2)

Where A = actual effort value and P = Predicted effort value

PRED(25) = m/n (3)

Here m is the count of observations with MMRE value less than or equal to 0.25 and n is the count of data points in the dataset.

Function software_effort_estimation(α, β, ϒ, £, ¥1, ¥2, ¥3, ¥4, ¥5)

Inputs:

- Three datasets α, β and ϒ with different types of attributes
- KNN missing value filter i.e. £
- Five training algorithms i.e. ¥1,¥2,¥3,¥4,¥

Outputs:

- €, mean magnitude of relative error
- Ω_{25}, number of data points with MMRE less than or equal to 0.25
- Ω_{50} , number of data points with MMRE less than or equal to 0.50
- Ω_{75}, number of data points with MMRE less than or equal to 0.75

Step I Data Preprocessing

- *size of 3 input datasets α, β and ϒ is n1, n2 and n3.*
- *α'=£(α), β'=£(β), ϒ'=£(ϒ); where α', β'and ϒ'are the clean datasets of α, β, ϒ respectively and assume sizes of α', β'and ϒ'be n1, n2 and n3 respectively*

Step II Training and Validation

- *divide each of α', β' and ϒ' in to k random subsamples where k=10*
  - *sizeof(subsample(α')) = n1/k;*
  - *sizeof(subsample(β')) = n2/k;*
  - *sizeof(subsample(ϒ')) = n3/k;*
- *let subsamples of α' be α'_{10}, α'_{11}, α'_{12}, α'_{13}, α'_{14}, α'_{15}, α'_{16}, α'_{17}, α'_{18} and α'_{19}*
- *let subsamples of β' be β'_{20}, β'_{21}, β'_{22} ,β'_{23}, β'_{24} , β'_{25} , β'_{26} , β'_{27}, β'_{28} and β'_{29}*
- *let subsamples of ϒ' be Υ'_{30} ,Υ'_{31}, Υ'_{32} ,Υ'_{33}, Υ'_{34}, Υ'_{35}, Υ'_{36} ,Υ'_{37} ,Υ'_{38} and Υ'_{39}*
- *for(i=0; i<count_evolutionaryalgorithms;i++) //where i=5*

```
{   train_and_validate (α');
train_and_validate(β');
train_and_validate(ϒ')
}
```

```
function train_and_validate (μ)
{
sum_estimated_effort=0, avg_estimated_effort, estimated_effort[10];
for (k=0; k<10;k++)
{
  train(μ1(k+1)%10, μ1(k+2)%10, μ1(k+3)%10, μ1(k+4)%10, μ1(k+5)%10, μ1(k+6)%10, μ1(k+7)%10, μ1(k+8)%10,
  μ1(k+9)%10 );
validate (μ1k);
estimated_effort [k] = Calculate_estimated_effort (k);
}
end for
for(k=0;k<10;k++)
 {
sum_estimated_effort=sum_estimated_effort+ Estimated_effort[k];
 }{
avg_estimated_effort= (sum_estimated_effort)/10;
estimated_effortk = avg_estimated_effort;
}
```

Step III: Calculation of Performance Measures

```
for k=1 to 3  // 3 datasets
{   €K = (actual_effortk- estimated_effortk) /actual_effortk;
   Ωk(25) = (count (datapoints_with_MRE<= 0.25))/nk;
   ΩK(50) =(count (datapoints_with_MRE<= 0.50))/nk
   ,Ω k(75) =(count (datapoints_with_MRE<= 0.75))/nk;
 }
end for
 }
```

Figure 3.1. Pseudocode for Estimating Software Effort

IV. SIMULATION ENVIRONMENT AND RESULTS

In this section, first the simulation environment used to estimate the effort has been described and then different performance metrics to analyze the behavior of different techniques on different types of data have been calculated.

A. Simulation Environment

The simulation environment consists of a software tool to assess evolutionary algorithms is called Knowledge Extraction based on Evolutionary Learning tool (abbreviated as KEEL) on Pentium IV processor with memory. KEEL is an open source java tool that contains a broad variety of traditional knowledge extraction algorithms, learning techniques based oncomputational intelligence, hybrid algorithms, statistical approaches, preprocessing and post processing methods etc.

B. Datasets

In this paper, two datasets from PROMISE (PRedictOr Models In Software Engineering) Software repository and one data set from Google codes have been taken. These three datasets have different sizes ranging from large to middle to small. The two datasets from PROMISE namely Desharnais(large size) and Maxwell(middle size) were taken as they cover broad range of attributes ranging from team experience, length of project, cohesion, coupling to source of project. In order to analyze different set of parameters, the third dataset, Miyazaki 94 (small size), was taken from google codes. The parameters of Miyazaki that have been worked on were total number of forms in a project, number of input & output screens, total number of data elements in files, total number of data elements in all screens, total number of files in the project etc**.** The characteristics of these datasets are given in Table 4.2.

Table 4.2. Characteristics of Datasets

Dataset	No. of observations (size)	No. of features (input variables)	No. of numerical features	No. of categorical features
Desharnais	82	7	7	0
Maxwell	62	25	3	22
Miyazaki94	48	8	7	1

C. Empirical Study and Results

The results obtained through empirical study for Desharnais, Maxwell and Miyazaki94 datasets are depicted in Figure 4.3.1, Figure 4.3.2 and Figure 4.3.3respectively. GFS-GPG-R has shown best performance for Desharnais dataset with MMRE value of 0.288. This kind of performance can be due to non dependency of this technique on error surface and ability to solve non-differential and non-continuous problems like software effort estimation. After that GFS-GSP-R and GFS-SAP-Sym-R have almost same MMRE value of 0.299 for this dataset.For Maxwell dataset, GFS-GSP-R is the best method in terms of MMRE value having lowest MMRE value of 0.273. Further GFS-SAP-Sym-R have also shown good performance with MMRE values of 0.284. As can be seen from Figure 4.3, GFS-SAP-Sym-R outperforms all other models in term of MMRE value for Miyazak94 dataset. It is having the lowest MMRE value of 0.27. After this GFS-GSP-R and FRSBMhave also shown good performance having MMRE values of 0.296 and 0.419 respectively.It can be seen that GFS-GSP-R has shown good performance for all three datasets. Combination of genetic programming, simulated annealing and fuzzy in GFS-GSP-R can be responsible for this kind of behaviour. For large and medium sized datasets, GFS-SAP-Sym-R is also a good model for predicting effort. It can be due to the fact that generalization capability of this method improves as the size of dataset increases.Ensemble-R method has shown worst performance for all three datasets with MMRE value of 0.594 for Desharnais, 1.174 for Maxwell and 0.77 for Miyazaki94 dataset. This may be due to less variety in the neural network models ensembled in ENSEMBLE-R method.Also, the results have been analyzed by calculating PRED(25) values across all three datasets for different evolutionary algorithms discussed. As can be seen from Table 4.3.1, GFS-GSP-R and GFS-SAP-Sym-R have highest values for PRED(25) for all fifteen combinations of evolutionary algorithms and datasets (5 evolutionary algorithms x 3 datasets). This kind of behaviour can be due to the fact that both GFS-GSP-R and GFS-SAP-Sym-R involves simulated annealing learning which guarantees finding an optimal solution.

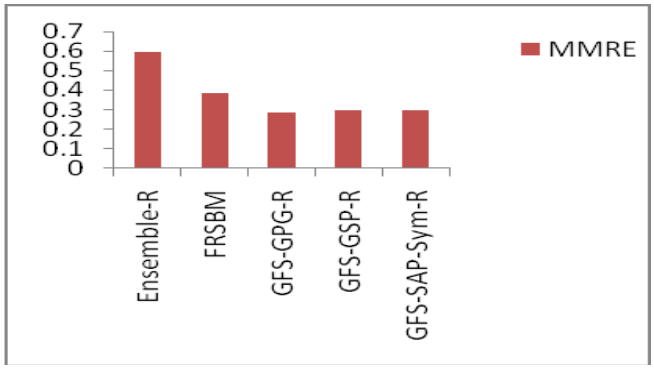

Figure 4.3.1. Results for Desharnais Dataset

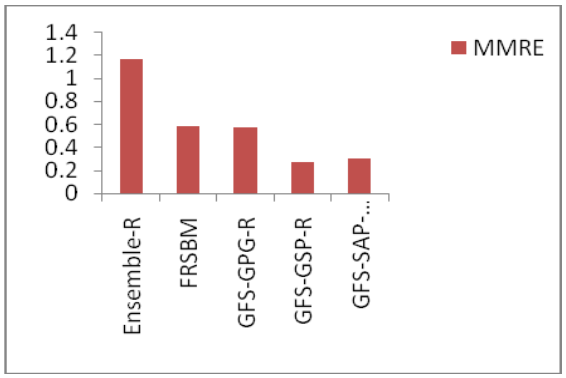

Figure 4.3.2. Results for Maxwell Dataset

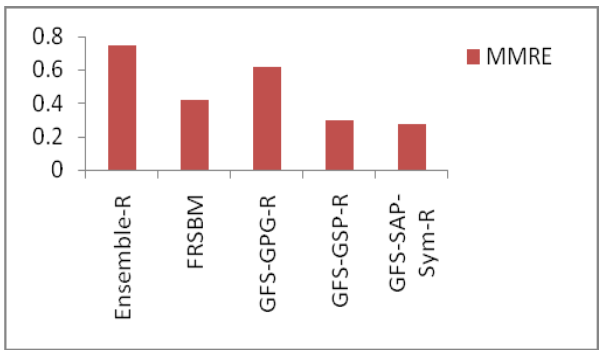

Figure 4.3.2. Results for Miyazaki94 Dataset

Also, the results obtained from empirical study in this paper have been compared with the previous results obtained from application of machine learning techniques for effort estimation on these datasets in [13]. It can be seen from Table 4.3.1 and Table 4.3.2 that evolutionary learning algorithms used in this paper are better than other machine learning algorithms like MLP, SVR and ANFIS for estimating software effort. This may be due to the fact that most of the algorithms used in the paper are hybridized (i.e. combination of two or more algorithms). Due to hybridization, the error in predicting effort minimizes.

Table 4.3.1. Results of Evolutionary Learning Algorithms

Dataset Used	Methods Used	MMRE	PRED(25)
Desharnais	Ensemble-R	0.594	0.16
	FRSBM	0.385	0.284
	GFS-GPG-R	0.288	0.346
	GFS-GSP-R	0.299	0.333
	GFS-SAP-Sym-R	0.299	0.333
Maxwell	Ensemble-R	1.174	0.129
	FRSBM	0.583	0.258
	GFS-GPG-R	0.573	0.113
	GFS-GSP-R	0.273	0.306
	GFS-SAP-Sym-R	0.299	0.306
Miyazaki94	Ensemble-R	0.75	0.125
	FRSBM	0.419	0.208
	GFS-GPG-R	0.615	0.167
	GFS-GSP-R	0.296	0.313
	GFS-SAP-Sym-R	0.273	0.333

Table 4.3.1. Results of Machine Learning Algorithms

Dataset Used	Methods Used	MMRE	PRED(25)
Desharnais	MLP	0.782	0.2
	SVR	0.738	0.333
	ANFIS	1.057	0.2
Maxwell	MLP	0.748	0.25
	SVR	1.013	0.333
	ANFIS	1.904	0.167
Miyazaki94	MLP	0.658	0.2
	SVR	1.083	0.3
	ANFIS	0.361	0.2

V. CONCLUSIONS

On the basis of entire study, it is concluded that evolutionary algorithms used in this paper give better resultsfor software effort estimation as compared to other machine learning methods like MLP, SVR and ANFIS. There are so many factors which affect the results of software effort estimation like software complexity, computer platform, fragmentation etc. As there are many evolutionary algorithms for software effort prediction, it's difficult to say which evolutionary technique is better than the other one. The results of our research have been summarized in Table 5.1.

Table 5.1. Comparison of Different Evolutionary Algorithms for Three Datasets

Dataset	**Size**	**Best Method**	**Good Method**	**Worst Method**
Desharnais	Large	GFS-GPG-R	GFS-GSP-R,GFS-SAP-Sym-R	Ensemble-R
Maxwell	Medium	GFS-GSP-R	GFS-SAP-Sym-R,	Ensemble-R
Miyazaki94	Small	GFS-SAP-Sym-R	GFS-GSP-R	Ensemble-R

After analyzing the results with different datasets and using different evolutionary algorithms on the datasets it can be concluded that different datasets show different results with different algorithms. The performance of a particular evolutionary algorithm depends on the type of data on which our evolutionary algorithm is trained. As per studies inthis paper, GFS-SAP-Sym-R is a good method for dataset of any size (small, medium and large). Out of 3 techniques, Ensemble-R has shown worst performance for all 3 datasets.Budget of a project can be estimated correctly if applicability of a particular evolutionary technique under various circumstances is known. As per studies in this paper, if the dataset is of small, medium or large size, selection of evolutionary technique can be done accordingly.Correct manpower estimations for a particular project can be made. Selection of correct software effort estimation technique can ensure timely completion of software projects. Also, quality of software projects improves if software estimation technique is

selected wisely. In the future, this empirical study can be done for some industrial software. Further, we can apply other evolutionary algorithms like bacterial foraging and particle swarm optimization on the same datasets and evaluate if there is any performance improvement in results or not. This study can be repeated again for other new techniques until we find a unique technique which can give accurate and correct estimation for all types of datasets.

REFERENCES

[1] Y. Singh, P.K. Bhatia, A. Kaur, and O. Sangwan, 'A Review of Studies on Effort Estimation Techniques of Software Development',in Proc. Conference Mathematical Techniques: Emerging Paradigms for Electronics and IT Industries, New Delhi, 2008, pp. 188-196.

[2] B. Boehm, C. Abts and S. Chulani, Software, 'Development Cost Estimation Approaches - A survey', in Annal of Software Engineering, vol. 10, 2000, pp. 177-205.

[3] C. Jones, 'Estimating Software Costs: Bringing Realism to Estimating (2nd Edition)', in McGraw-Hill, 2007, New York Chicago.

[4] J. Li, G. Ruhe, A. Al-Emran and M. M. Richter, 'A Flexible Method for Software Effort Estimation by Analogy',in Empirical Software Engineering, vol. 12, 2006, pp. 65-106.

[5] J. Kaur, S. Singh and K. S. Kahlon, ' Comparative Analysis of the Software Effort Estimation Models', in Proc. World Academy of Science, Engineering and Technology, Auckland, New Zealand, 2008, pp. 485-487.

[6] I. Attarzadeh and S. H. Ow, 'Software Development Effort Estimation Based on a New Fuzzy Logic Model', in International Journal of Computer Theory and Engineering, vol. 1, 2009 pp. 1793-8201.

[7] M. Jorgensen, 'A review of studies on expert estimation of software development effort', in The Journal of Systems and Software, vol. 70, 2004, pp. 37-60.

[8] M. Jordensen, G. Kirkeboen, D. I. K. Sjoberg, B. Anda and L. Bratthall, 'Human Judgement in Effort Estimation of Software Projects', in Proc. International Conference on Software Engineering, Limerick, Ireland, 2000.

[9] C. J. Burgess and M. Le, 'Can genetic programming improve software effort estimation ? A comparative evaluation', in Information and Software Technology, 2001, vol. 43.

[10] S. Amasaki, 'A Study on Performance Inconsistency between Estimation by Analogy and Linear Regression',in 'SEKE' , Knowledge Systems Institute Graduate School, 2011, pp 485-488 .

[11] U. Keel, M. Graczyk, T. Lasota, and B. Trawiński, 'Comparative Analysis of Premises Valuation Models'in proceedings ICCCI '09 Proceedings of the 1st International Conference on Computational Collective Intelligence. Semantic Web, Social Networks and Multiagent Systems, 2009, pp 800-812.

[12] M. O. Elish, 'Improved estimation of software project effort using multiple additive regression trees', in Expert Syst. Appl., vol. 36, no. 7, Sep 2009', pp. 10774–10778.

[13] M. O. Elish, T. Helmy and M.I. Hussain, 'Empirical Study of Homogeneous and Heterogeneous EnsembleModels for Software Development Effort Estimation',in Mathematical Problems in Engineering , vol. 2013 , 21.